

Log de Construção — Forja App (Backend C#/.NET)

Registro por fase, seguindo o método quadrangular (Contexto → Porquê → Desenvolvimento isolado → Teste isolado). Referência: `especificacao-app-concursos.md` e `plano-arquitetural-backend-csharp.md`.

Nota de contexto: o projeto começou com um protótipo em Next.js/Prisma (ingestão de conteúdo, curadoria de questões, testes de infraestrutura). Esse repositório foi descartado por completo — o único remanescente é o banco de dados já populado no Neon (carreiras, bancas, editais, tópicos, chunks de conteúdo, questões). Este log registra a construção do backend em C#/.NET a partir da criação do Domain, que é o que está em andamento agora.

Arquitetura: Frontend (Next.js/PWA) → Core API (ASP.NET Core, Clean Architecture) → Postgres/Neon + pgvector ← Motor de Conteúdo (Python). Plano completo em `plano-arquitetural-backend-csharp.md`.

Skill adotada: `.NET/C# Best Practices` (DI via primary constructor, interfaces prefixadas com `I`), testes MSTest/FluentAssertions/Moq a partir da Fase 2). Semantic Kernel escopado só na Fase 5 (Motor RAG) — justificado pelo histórico real de troca de provedor de IA no projeto anterior.

Repositório: `github.com/mtpachecoo/forja-app`.

Fase 1 — Setup da Solução e Domain (✅ concluído)

Contexto: repositório novo, zero dependência de infraestrutura ainda.

Desenvolvimento:

- Solution com 4 projetos: `Forja.Domain`, `Forja.Application`, `Forja.Infrastructure`, `Forja.Api` (últimos 3 vazios, prontos pra próximas fases)
- 18 entidades como POCOs puros + 6 enums (com XML doc citando o valor exato do Postgres em cada membro, pra conferência 1:1 com o DDL)
- `IRepository<TEntity, TKey>` genérico (não `IRepository<T>` fixo em `Guid`) — ajuste identificado pelo próprio Claude Code, já que `Streak`/`Pontuacao` são chaveadas por `UsuarioId` e `UsuarioMedalha` tem chave composta
- **Decisão de modelagem:** `Questao.Alternativas` como `List<Alternativa>?` (record `Alternativa(string Letra, string Texto)`), não `List<string>` nem `Dictionary<string, string>` — elimina a ambiguidade de ordem/posição que o JSONB bruto não garante
- Bug pego antes do commit: `Questao.cs` e `Alternativa.cs` existiam só como buffer não salvo no editor, não no disco — Claude Code detectou ao tentar referenciar o arquivo e corrigiu antes de commitar

Teste isolado: build com 0 erros, conferência manual das entidades/enums contra o DDL do banco já existente no Neon.

Versionamento: `git init`, `.gitignore` (`bin/obj/.vs/.idea`), commit único, remote `origin` → `github.com/mtpachecoo/forja-app`, branch `main` publicada.

Fase 2 — EF Core e Infrastructure (✅ concluído)

Contexto: Domain completo (Fase 1), banco real no Neon com dado de produção.

Desenvolvimento: `ForjaDbContext` com 19 `DbSet`, `.ToTable()` explícito por `Configuration` (pegou irregularidade `revisao_espacada` no singular), `EFCore.NamingConventions` pra `snake_case` automático, conversor de URI do Neon pro formato `Npgsql` (sem assumir comportamento não documentado), `AlternativasJsonConverter` + `ValueComparer`, `EmbeddingVectorConverter`, 7 repositórios concretos, `BuscarPorSimilaridadeAsync` via `FromSqlInterpolated` parametrizado com distância de cosseno (`<=>`).

Decisão registrada: `Repository<T>` não chama `SaveChangesAsync` — persistência via `IUnitOfWork`/serviço da Application, pendente pra Fase 4 (atomicidade multi-tabela em `POST /respostas`).

Teste automatizado: 27/27 passando (24 unitários + 3 smoke test contra o Neon real).

Bugs reais pegos pelo smoke test (só apareceriam em produção):

1. `HasPostgresEnum` no `modelBuilder` sozinho não bastava — exigia `MapEnum<T>()` no builder do `UseNpgsql` (padrão EF9+)
2. `HasColumnType("vector(1024)")` explícito conflitava com o plugin do `pgvector` — removido, `.UseVector()` resolve sozinho
3. `alternativas` no banco é objeto JSON (`{"A": "...", "B": "..."}`), não array de `{Letra, Texto}` — o converter faz a ponte, Domain continua limpo

Achado de licenciamento: `FluentAssertions` puxou 8.10.0 (licença comercial paga da Xceed) por padrão — rebaixado pra `7.2.2` (última MIT). **Pendência:** fixar essa versão explicitamente no `.csproj` pra não voltar a puxar 8.x num restore futuro.

Fase 3 — Módulo de Autenticação (✅ concluído)

Contexto: Infrastructure completo (Fase 2), schema `neon_auth` já existe e é gerenciado pelo Neon.

Desenvolvimento: validação de JWT via `ScottBrady.IdentityModel` (`EdDSA/Ed25519` — `Microsoft.IdentityModel.Tokens` nativo não suporta esse algoritmo), `IUsuarioService` com provisionamento no primeiro acesso, `GET /me` (primeiro código real em `Forja.Api`).

Descobertas via pesquisa/empirismo:

- Namespace real do pacote é `ScottBrady.IdentityModel.*`, não `ScottBrady91.*` como a documentação sugeria — descoberto via reflection num projeto descartável
- `.env`/`.env.example` atualizados com `NeonAuth_BaseUrl`

Teste automatizado: 32/32 passando (27 de Infrastructure + 5 de `UsuarioServiceTests`): usuário novo cria e persiste, usuário existente retorna sem persistir, sem claim `sub` rejeita, claim `sub` inválida rejeita, sem identidade externa rejeita).

Verificação manual (curl): sem `Authorization` → 401; token malformado → 401; token bem-formado com assinatura forjada → 401, confirmando que o pipeline buscou o JWKS real, achou a chave pelo `kid`, converteu pra `EdDsaSecurityKey` e rejeitou a assinatura — valida o caminho crítico ponta a ponta.

Validado de ponta a ponta com token real: `GET /me` retornou 200 com o perfil correto; registro em `public.usuarios` criado no primeiro acesso (provisionamento automático confirmado).

Descobertas do processo (documentadas para não perder tempo de novo):

- `signup`/`signin` no Neon Auth exigem `callbackURL` absoluto
- A página de teste tem que rodar num navegador de verdade, não no preview da IDE (Origin real necessário)
- `Forja.Api` não carrega `.env` sozinha (só os testes têm isso via `EnvFile.cs`) — precisa exportar as env vars antes do `dotnet run`
- No PowerShell, `curl` é alias de `Invoke-WebRequest` (não aceita `-H`) — usar `curl.exe` ou `Invoke-RestMethod`

`tools/auth-test/` mantido no repo (ignorado pelo Git) como ferramenta de debug para gerar token real rapidamente no futuro.

Fase 4 — Core de Questões (✅ concluído)

Contexto: Infrastructure e Auth completos e testados (Fases 2-3). `Repository<T>` não chama `SaveChangesAsync` — pendência da Fase 2 a resolver aqui.

Desenvolvimento:

- `IUnitOfWork` com `SaveChangesAsync` único, chamado uma vez por caso de uso (não mais por repositório) — resolve a pendência da Fase 2, garantindo atomicidade quando `RespostaUsuario` e `Pontuacao` são gravados juntos
- `QuestaoService` (filtro por carreira/banca/disciplina, independentes — RF-004) e `RespostaService` (RN-004, RN-008 a RN-012)
- `NotFoundException` genérica (`Forja.Application.Common`, `(string entidade, object id)`) — substituiu uma exceção nomeada por entidade que tinha sido criada inicialmente, evitando inflar o projeto com uma classe por entidade
- Endpoints: `GET /questoes` (com filtro), `GET /questoes/{id}`, `POST /respostas`
- **Refatoração do middleware de exceção:** lógica movida de dentro de `Program.cs` pra `Forja.Api/ExceptionHandling/GlobalExceptionHandler.cs` (`ExceptionHandler`), com `UsuarioNaoAutenticadoException` → 401 incluído (identificado pelo próprio Claude Code como necessário pra manter a consolidação coerente, não estava na lista

original). `Program.cs` ficou só com composição (`AddExceptionHandler`, `AddProblemDetails`, `UseExceptionHandler`). Formato de resposta de erro passou a `ProblemDetails` padrão do ASP.NET Core (RFC 7807) — mudança inerente ao padrão, não escolha extra

Teste automatizado: 15/15 passando (`QuestaoServiceTests`, `RespostaServiceTests`, `UsuarioServiceTests`) antes do refactor de middleware; comportamento reconfirmado manualmente via curl depois do refactor (sem token → 401, token forjado → 401).

Pendência de ambiente (isolada, não bloqueia o roadmap): Smart App Control do Windows bloqueia `Forja.Application.Tests.dll` recém-compilada (política de reputação, não antivírus/malware) — contorno momentâneo via Test Explorer do Visual Studio/Rider; WSL2 é alternativa se o contorno parar de funcionar. Causa raiz não resolvida, só isolada.

Auditoria SOLID pós-Fase 4 (✅ concluída): revisão estruturada (SRP, DIP, ISP, LSP, peso morto, composição do `Program.cs`) — DIP/ISP/LSP e `Program.cs` sem achados. Dois itens corrigidos:

1. `RespostaService` dependia de `IQuestaoRepository` direto e duplicava a checagem "questão aprovada" que já existia em `QuestaoService` — corrigido para depender de `IQuestaoService`, duplicação eliminada
2. SRP: lógica de pontuação/virada de semana extraída do `RespostaService` para `Pontuacao.RegistrarPontos(int pontos, DateOnly hoje)`, no próprio agregado — comportamento em memória, sem I/O; persistência continua via `IUnitOfWork`, fora da entidade

Itens sinalizados e conscientemente não corrigidos: registro morto de `IRepository<, >` genérico (baixa prioridade), repositórios ainda não consumidos (`PlanoEstudo`, `SessaoEstudo`, `RevisaoEspacada`, `ChunkConteudo`) — aguardando fases futuras), interfaces com implementação única (decisão arquitetural deliberada, já aproveitada pelos testes com mock).

Teste automatizado final: 18/18 em `Forja.Application.Tests` (incluindo `PontuacaoTests` novo, cobrindo o rollover semanal), 27/27 em `Forja.Infrastructure.Tests`.

Fase 5 — Motor RAG (✅ concluído)

Contexto: Fase 4 completa e auditada. Modelo de embedding confirmado: `voyage-3`, 1024 dimensões (mesmo usado nos 1.005 chunks já indexados antes do pivot — crítico bater exatamente, embeddings de modelos diferentes não são comparáveis por cosseno mesmo com dimensão igual).

Desenvolvimento:

- `Forja.Infrastructure.Ia` como projeto separado de `Forja.Infrastructure` — isola o churn de pacotes de IA instáveis (`Microsoft.SemanticKernel.Connectors.Google` em alpha, `1.78.0-alpha`) da persistência estável (EF Core/Npgsql)

- `Forja.Application/Duvidas/`: `IRagService`, `IGeradorDeEmbeddings` (porta nova), `RagService` — depende de `IChatCompletionService` (abstração do Semantic Kernel), não do `Kernel` inteiro, mantendo mockável
- `VoyageEmbeddingService`: sem conector oficial do Semantic Kernel pra Voyage, implementado via `HttpClient` puro (REST direto), documentado no código
- `RagService.ResponderDuvidaAsync`: valida questão via `IQuestaoService` (reaproveita `NotFoundException`, sem duplicar checagem de Aprovada), embeba a pergunta, busca 5 chunks mais similares, monta prompt com trechos numerados, grounding estrito.
Zero chunks retornados → mensagem fixa sem chamar o LLM (evita depender só da instrução de prompt pra não alucinar)
- `POST /duvidas` em `Forja.Api`, composição limpa (sem lógica inline)
- Config: `Gemini__Modelo=gemini-2.5-flash`, `Voyage__Modelo=voyage-3`, chaves via `.env`

Teste automatizado: 3 testes novos em `RagServiceTests`, capturando o `ChatHistory` real passado ao LLM e confirmando que o texto dos chunks mockados aparece no prompt, mais o caso de zero chunks. Build limpo (7 projetos): 21/21 Application, 27/27 Infrastructure.

Risco sinalizado antes do teste manual: `gemini-2.5-flash` já teve problema de disponibilidade neste projeto (com outra chave, no protótipo anterior) — testar disponibilidade real com a chave atual, não assumir.

Pendência: teste manual (`POST /duvidas` com token real e questão de Direito aprovada), a cargo do usuário — confirmar que a resposta cita algo do `ChunksUsadosIds` retornado.

Próximos módulos — ainda não iniciados

- Fase 5 — Motor RAG (com Semantic Kernel)
- Fase 6 — Planejamento, Ciclos e Gamificação